Summarize

Part III. Serving Patterns

Chapter 5 described patterns for grouping collections of containers that are scheduled on the same machine. These groups are tightly coupled, symbiotic systems. They depend on local, shared resources like disk, network interface, or inter-process communications. Such collections of containers are important patterns, but they are also building blocks for larger systems. Reliability, scalability, and separation of concerns dictate that real-world systems are built out of many different components, spread across multiple machines. In contrast to single-node patterns, the multinode distributed patterns are more loosely coupled. While the patterns dictate patterns of communication between the components, this communication is based on network calls. Furthermore, many calls are issued in parallel, and systems coordinate via loose synchronization rather than tight constraints.

Recently, the term *microservices* has become a common term for describing multinode distributed software architectures. Microservices describe a system built out of many different components running in different processes and communicating over defined APIs. Microservices stand in contrast to *monolithic* systems, which tend to place all of the functionality for a service within a single, tightly coordinated application. These two different architectural approaches are shown in Figures III-1 and III-2.

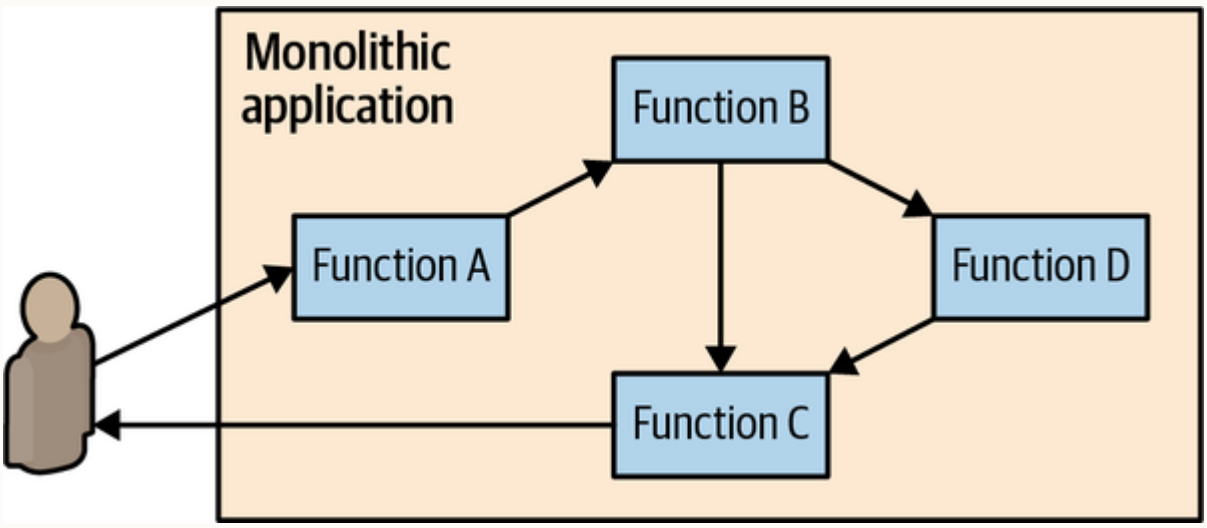


Figure III-1. A monolithic service with all functions in a single container

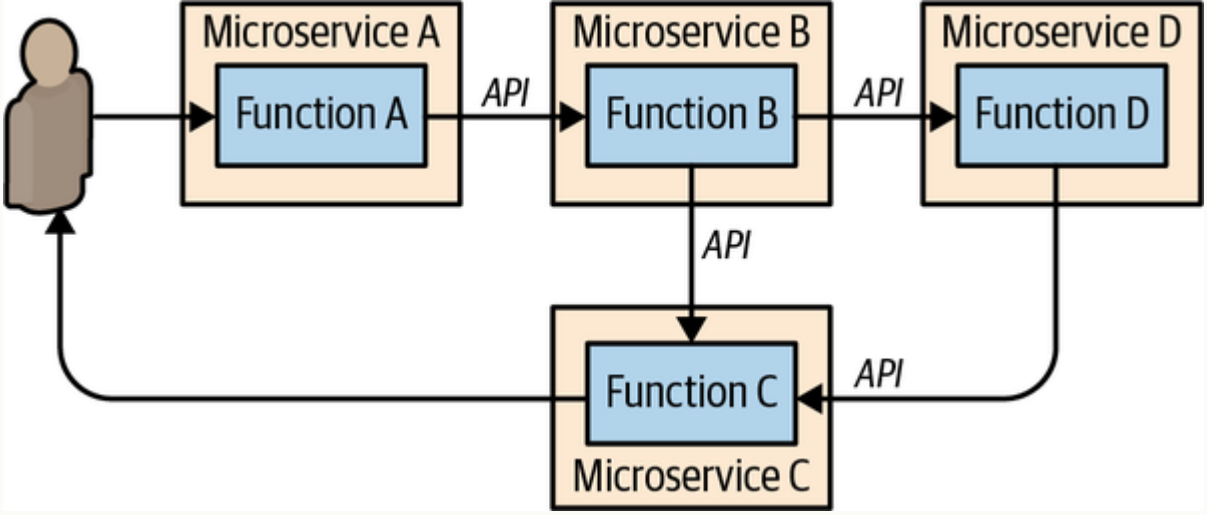


Figure III-2. A microservice architecture with each function broken out as a separate microservice

There are numerous benefits to the microservices approach—most of them are centered around reliability and agility. Microservices break down an application into small pieces, each focused on providing a single service. This reduced scope enables each service to be built and maintained by a single "two pizza" team. Reduced team size also reduces the overhead associated with keeping a team focused and moving in one direction.

Additionally, the introduction of formal APIs in between different microservices decouples the teams from one another and provides a reliable contract between the different services. This formal contract reduces the need for tight synchronization among the teams because the team providing the API understands the surface area that it needs to keep stable, and the team consuming the API can rely on a stable service without worrying about its details. This decoupling enables teams to independently manage their code and release schedules, which in turn improves each team's ability to iterate and improve their code.

The design of a service architecture is much like the design of a class hierarchy—its goals are to introduce abstraction, encapsulation, and modularity. It is important to note that in each of these categories the "right" design has as much to do with the humans involved in building the design as it does with the core application being developed.

Abstraction and encapsulation can often be seen as foils of each other. Abstraction enables teams to not worry about the details of how a complex piece of functionality is implemented and focus instead on what they can do with that capability. The programming required to recognize objects in an image using machine learning is quite complicated, but the abstract service `findObjectsInImage(image): objects` is quite easy to use. Distributed systems are quite complex, and trying to keep all of the details in our heads rapidly leads to information overload. Abstraction enables teams to focus on higher-level capabilities, not lower-level implementation details.

Encapsulation is the same but in reverse; encapsulation allows teams to deliver a service while hiding the implementation details from the user. Hiding these details is critical because it enables the implementation team to make changes without breaking their users. This flexibility makes teams more agile because they are decoupled from each other. Taken together, encapsulation and abstraction enable us to build optimally sized teams for execution and delivery. Big enough to take on hard problems with a sense of purpose, but small enough to move quickly without the communication baggage that arises in larger teams.

Finally, the decoupling of microservices enables better scaling. Because each component has been broken out into its own service, it can be scaled independently. It is rare for each service within a larger application to grow at the same rate, or have the same way of scaling. Some systems are stateless and can simply scale horizontally, whereas other systems maintain state and require sharding or other approaches to scale. By separating each service out, each service can use the approach to scaling that suits it best. This is not possible when all services are part of a single monolith.

But of course there are downsides to the microservices approach to system design as well. As the system has become more loosely coupled, debugging the system when failures occur is significantly more difficult. You can no longer simply load a single application into a debugger and determine what went wrong. Any errors are the by-products of a large number of systems often running on different machines. This environment is quite challenging to reproduce in a debugger. As a corollary, microservices-based systems are also difficult to design and architect. A microservices-based system uses multiple methods of communicating between services; different patterns (e.g., synchronous, asynchronous, message-passing, etc.); and multiple different patterns of coordination and control among the services.

Additionally, there can be a tendency within teams to take a microservice pattern to an extreme and end up with far too many microservices that are far too small. This oversegmentation not only adds complexity, but it also can add significant inefficiency in terms of both network overhead (since communication between microservices occurs on the network) and also costs (since each instance of a microservice has a fixed amount of overhead in terms of CPU and memory). As a general rule of thumb, if you have more microservices than you have engineers on your team, you either have a very small team or you are overdoing the microservice approach.

These challenges are the motivation for distributed patterns. If a microservices architecture is made up of well-known patterns, then it is easier to design because many of the design practices are specified by the patterns. Additionally, patterns make the systems

easier to debug because they enable developers to apply lessons learned across a number of different systems that use the same patterns.

With that in mind, this section introduces a number of multinode patterns for building distributed systems. These patterns are not mutually exclusive. Any real-world system will be built from a collection of these patterns working together to produce a single higher-level application.

Previous chapter
< 5. Adapters

Next chapter
6. Replicated Load-Balanced Services >